loader.py — technical

`src/moe_pruning/model/loader.py · 198 lines · moe_pruning.model.loader`

Purpose

Model, tokenizer and plugin loading with placement; lazy access to original checkpoint tensors.

Step by step

Classes and functions in source order. Signatures, line ranges, raised exceptions and settings are extracted from the code; numbered steps describe what the code does.

`class LoadedModel`

lines 24-31 · dataclass

The loaded model, tokenizer, input device, dtype, source and plugin.

Field	Type	Default
`model`	PreTrainedModel	
`tokenizer`	PreTrainedTokenizerBase	
`device`	torch.device	
`dtype`	torch.dtype	
`source`	str	
`revision`	str | None	
`plugin`	object	

`_source`

```
def _source(cfg: ExperimentConfig) -> tuple[str, str | None]
```

lines 34-38 · settings HF_MODEL_ID

1. `(MODEL_PATH, None)` if set, else `(HF_MODEL_ID, HF_REVISION)`.

`hub_kwargs`

```
def hub_kwargs(cfg: ExperimentConfig) -> dict
```

lines 41-47 · settings HF_CACHE_DIR

`revision` / `cache_dir` for every Hugging Face download (HF_CACHE_DIR).

1. `revision` plus `cache_dir=HF_CACHE_DIR` when configured; passed to every Hub call.

`model_load_kwargs`

```
def model_load_kwargs(cfg: ExperimentConfig) -> dict
```

lines 50-55 · settings EXPERTS_IMPLEMENTATION

Extra `from_pretrained` kwargs common to all model loads.

1. `hub_kwargs` plus `experts_implementation` when configured.

`load_tokenizer`

```
def load_tokenizer(cfg: ExperimentConfig) -> PreTrainedTokenizerBase
```

lines 58-64

Load the tokenizer (sets a pad token if the model has none).

1. `AutoTokenizer.from_pretrained`; if no pad token, use EOS as pad (padding is always masked).

`get_model_plugin`

```
def get_model_plugin(cfg: ExperimentConfig)
```

lines 67-73

Model plugin of the configured model (reads only its config.json).

`load_original_model`

```
def load_original_model(cfg: ExperimentConfig, device: torch.device | None=None) -> LoadedModel
```

lines 76-107 · settings MODEL_DTYPE

Load the unmodified model in evaluation mode on its configured device(s).

1. Build the placement from config (an explicit `device` argument overrides the primary device).
2. Resolve dtype, warn on poor combinations, log device and placement.
3. Find the plugin from `config.json`; `plugin.auto_model_class.from_pretrained(source, dtype=..., **model_load_kwargs, **from_pretrained_kwargs(placement))` - with a device map, weights are loaded straight onto their GPUs.
4. Single device: `model.to(device)`. Set eval mode; the returned device is the input embedding's device.

`class CheckpointReader`

lines 110-151

Lazy, memory-mapped access to the tensors of the ORIGINAL checkpoint on disk.

`CheckpointReader.__init__`

```
def __init__(self, model_dir: Path)
```

lines 119-139 · raises FileNotFoundError

1. If the directory has `*.safetensors`: open each with `safe_open` and index every tensor name to its handle.
2. Else: `torch.load(..., mmap=True, weights_only=True)` each `pytorch_model*.bin` and index names.
3. No files -> `FileNotFoundError`. `get(name)` reads only that tensor (clone for .bin).

`CheckpointReader.keys`

```
def keys(self) -> list[str]
```

lines 141-142

Names of all tensors in the checkpoint.

`CheckpointReader.has`

```
def has(self, name: str) -> bool
```

lines 144-145

True if a tensor with this name exists.

`CheckpointReader.get`

```
def get(self, name: str) -> torch.Tensor
```

lines 147-151

Read one tensor.

`open_original_checkpoint`

```
def open_original_checkpoint(cfg: ExperimentConfig) -> CheckpointReader
```

lines 154-164

CheckpointReader for the configured original model (local dir or HF cache).

1. Local `MODEL_PATH` -> reader on it; otherwise `snapshot_download` restricted to weights/config files (uses the cache, downloads only if missing).

`class StateDictReader`

lines 167-180

CheckpointReader over an in-memory state dict (tests, already-loaded weights).

`StateDictReader.keys`

```
def keys(self) -> list[str]
```

lines 173-174

Names of all tensors.

`StateDictReader.has`

```
def has(self, name: str) -> bool
```

lines 176-177

True if a tensor with this name exists.

`StateDictReader.get`

```
def get(self, name: str) -> torch.Tensor
```

lines 179-180

Copy of one tensor.

`load_pruned_for_config`

```
def load_pruned_for_config(cfg: ExperimentConfig, model_path: Path)
```

lines 183-198

`load_pruned_model` with the configured placement, cache dir and base model.

1. Call `load_pruned_model` with the configured placement, the current base model source and load kwargs; return the model, tokenizer, manifest and input device.

Errors raised

Function	Exception	Message (f-string fields in braces)
`__init__`	FileNotFoundError	No checkpoint files (*.safetensors / pytorch_model*.bin) in {self.model_dir}